

Capítulo 21

Programação Orientada a Objetos: Polimorfismo

Fazendo a Diferença — EmissaoCarbono e o Clima

*“Este exercício combina o conceito técnico do capítulo — classes abstratas com somente funções virtuais puras — com a reflexão social sobre emissões de carbono. Três classes não relacionadas por herança normal (*Predio*, *Carro*, *Bicicleta*) compartilham o contrato de uma classe abstrata pura *EmissaoCarbono*. É o exemplo protótipo da diferença entre ‘herança de interface’ (Ex. 21.5) e herança de implementação: aqui não há código compartilhado, só um contrato de comportamento.”*

1 Exercício 21.17 — Classe abstrata *EmissaoCarbono*: polimorfismo

Enunciado. Usando uma classe abstrata com apenas funções virtuais puras, você pode especificar comportamentos semelhantes de classes possivelmente divergentes. Governos e empresas estão cada vez mais preocupados com as emissões de carbono. Crie três classes pequenas não relacionadas por herança: *Predio*, *Carro* e *Bicicleta*. Escreva uma classe abstrata *EmissaoCarbono* com apenas um método virtual puro *retEmissaoCarbono*. Faça com que cada uma de suas classes herde dessa classe abstrata e implemente o método. Crie objetos das três classes, coloque-os em um vetor de ponteiros *EmissaoCarbono* e percorra polimorficamente.

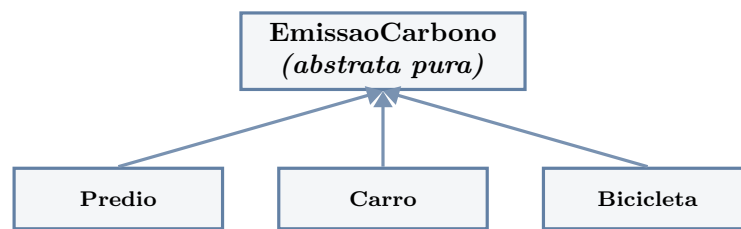
Análise do problema

Três objetos sem *nenhuma* relação natural em termos de herança:

- *Predio*: tem endereço, consumo elétrico, gera emissão indireta via geração de energia.
- *Carro*: tem modelo, quilometragem, consumo de combustível, gera emissão direta por queima.
- *Bicicleta*: tem modelo, vida útil; gera emissão quase nula em uso, apenas pela fabricação amortizada.

O que *compartilham*? **Um conceito abstrato:** todos podem responder *quanto CO₂ emitem por ano*. Esse é o ponto exato de injeção do polimorfismo — uma função virtual pura no nível mais alto possível.

Estrutura proposta



A **essência da modelagem**: a herança aqui não expressa nenhuma relação real “é-um” (uma bicicleta não *é* uma emissão de carbono). É mais como “cumpre o contrato de” — equivalente a uma interface em Java/C#.

Classe abstrata EmissaoCarbono

```

1 // Exercício 21.17 - EmissaoCarbono.h
2 // Classe abstrata com APENAS funcoes virtuais puras.
3 // Cada classe que herda deve implementar retEmissaoCarbono().
4 #ifndef EMISAOCARBONO_H
5 #define EMISAOCARBONO_H
6
7 #include <string>
8
9 class EmissaoCarbono {
10 public:
11     virtual ~EmissaoCarbono() {}
12
13     // Retorna a emissao anual de CO2 em kg
14     virtual double retEmissaoCarbono() const = 0;
15
16     // Para identificacao na impressao
17     virtual std::string descricao() const = 0;
18 };
19
20 #endif
  
```

Classe Predio

```

1 // Exercício 21.17 - Predio.h
2 // Predio: emissao baseada em consumo eletrico anual.
3 #ifndef PREDIO_H
4 #define PREDIO_H
5
6 #include "EmissaoCarbono.h"
7 #include <string>
8
9 class Predio : public EmissaoCarbono {
10 public:
11     Predio(const std::string &endereco,
12            double consumoEletricoAnualKWh);
13
14     void setConsumo(double c);
15     double getConsumo() const;
16     std::string getEndereco() const;
17
18     // Fator US: ~0.4 kg CO2 por kWh (EPA, media de rede eletrica EUA)
19     double retEmissaoCarbono() const override;
20     std::string descricao() const override;
21
  
```

```

22 private:
23     std::string endereco;
24     double consumoEletricoAnualKWh;
25 };
26
27 #endif

1 // Exercício 21.17 - Predio.cpp
2 #include <sstream>
3 #include "Predio.h"
4 using namespace std;
5
6 Predio::Predio(const string &e, double c) : endereco(e) {
7     setConsumo(c);
8 }
9
10 void Predio::setConsumo(double c) {
11     consumoEletricoAnualKWh = (c >= 0) ? c : 0;
12 }
13 double Predio::getConsumo() const { return consumoEletricoAnualKWh; }
14 string Predio::getEndereco() const { return endereco; }
15
16 double Predio::retEmissaoCarbono() const {
17     // ~0.4 kg de CO2 por kWh (fator médio EUA)
18     return consumoEletricoAnualKWh * 0.4;
19 }
20
21 string Predio::descricao() const {
22     ostringstream oss;
23     oss << "Predio em " << endereco
24         << " (consumo " << consumoEletricoAnualKWh << " kWh/ano)";
25     return oss.str();
26 }

```

Fator de emissão adotado

A fórmula $\text{consumoKWh} * 0.4 \text{ kg de CO}_2$ é uma aproximação da média americana (EPA: cerca de 0,4 kg CO₂/kWh para a matriz elétrica média dos EUA, que mistura carvão, gás natural e fontes limpas). A média brasileira é *muito menor* (cerca de 0,07 kg/kWh, graças à hidreletricidade); porém, para um exercício comparável ao livro original, mantemos o fator EPA.

Classe Carro

```

1 // Exercício 21.17 - Carro.h
2 // Carro: emissao baseada em km rodados / consumo (km por litro).
3 #ifndef CARRO_H
4 #define CARRO_H
5
6 #include "EmissaoCarbono.h"
7 #include <string>
8
9 class Carro : public EmissaoCarbono {
10 public:
11     Carro(const std::string &modelo,
12           double kmPorAno, double kmPorLitro);
13
14     std::string getModelo() const;
15     double getKmPorAno() const;
16     double getKmPorLitro() const;
17 }

```

```

18 // ~2.3 kg de CO2 por litro de gasolina queimado
19 double retEmissaoCarbono() const override;
20 std::string descricao() const override;
21
22 private:
23     std::string modelo;
24     double kmPorAno;
25     double kmPorLitro;
26 };
27
28 #endif

```

```

1 // Exercício 21.17 - Carro.cpp
2 #include <sstream>
3 #include "Carro.h"
4 using namespace std;
5
6 Carro::Carro(const string &m, double kma, double kpl)
7     : modelo(m),
8       kmPorAno(kma >= 0 ? kma : 0),
9       kmPorLitro(kpl > 0 ? kpl : 1) {}
10
11 string Carro::getModelo() const { return modelo; }
12 double Carro::getKmPorAno() const { return kmPorAno; }
13 double Carro::getKmPorLitro() const { return kmPorLitro; }
14
15 double Carro::retEmissaoCarbono() const {
16     double litrosAno = kmPorAno / kmPorLitro;
17     return litrosAno * 2.3; // 2.3 kg CO2 por litro de gasolina
18 }
19
20 string Carro::descricao() const {
21     ostringstream oss;
22     oss << "Carro " << modelo
23         << " (" << kmPorAno << " km/ano, "
24         << kmPorLitro << " km/L)";
25     return oss.str();
26 }

```

Fator de emissão adotado

Uma queima completa de 1 litro de gasolina libera aproximadamente 2,3 kg de CO₂ (resultado da combustão estequiométrica de octano com oxigênio atmosférico, valor confirmado por órgãos como EPA e IPCC). Multiplicando pelo número de litros consumidos por ano (km/ano ÷ km/L), obtemos a emissão anual de cada carro.

Classe Bicicleta

```

1 // Exercício 21.17 - Bicicleta.h
2 // Bicicleta: emissao basicamente ZERO em uso.
3 // Modelamos a emissao de fabricacao amortizada pela vida util.
4 #ifndef BICICLETA_H
5 #define BICICLETA_H
6
7 #include "EmissaoCarbono.h"
8 #include <string>
9
10 class Bicicleta : public EmissaoCarbono {
11 public:
12     Bicicleta(const std::string &modelo, int vidaUtilEmAnos);

```

```

13
14     std::string getModelo() const;
15     int getVidaUtil() const;
16
17     // ~250 kg CO2 para fabricar uma bicicleta moderna,
18     // amortizada pela vida util => emissao "anual" pequena
19     double retEmissaoCarbono() const override;
20     std::string descricao() const override;
21
22 private:
23     std::string modelo;
24     int vidaUtilEmAnos;
25 };
26
27 #endif

```

```

1 // Exercício 21.17 - Bicicleta.cpp
2 #include <sstream>
3 #include "Bicicleta.h"
4 using namespace std;
5
6 Bicicleta::Bicicleta(const string &m, int v)
7     : modelo(m), vidaUtilEmAnos(v > 0 ? v : 1) {}
8
9 string Bicicleta::getModelo() const { return modelo; }
10 int Bicicleta::getVidaUtil() const { return vidaUtilEmAnos; }
11
12 double Bicicleta::retEmissaoCarbono() const {
13     // 250 kg CO2 de fabricacao, amortizado pela vida util
14     return 250.0 / vidaUtilEmAnos;
15 }
16
17 string Bicicleta::descricao() const {
18     ostringstream oss;
19     oss << "Bicicleta " << modelo
20         << " (vida util " << vidaUtilEmAnos << " anos)";
21     return oss.str();
22 }

```

Por que uma bicicleta tem emissão maior que zero?

Em *uso*, a bicicleta é literalmente zero-emissão (alimentada pelo próprio ciclista). Mas a sua *fabricação* consome energia e matérias-primas, e isso resulta em uma “pegada” de aproximadamente 250 kg de CO₂ por bicicleta produzida (estudos europeus). Para uma comparação justa, amortizamos esse custo pela vida útil:

$$\text{Emissão anual} = \frac{250 \text{ kg CO}_2}{\text{vida útil em anos}}.$$

Para uma bicicleta com vida útil de 15 anos: $250/15 \approx 16,7$ kg/ano. Compare com um carro pequeno (centenas de kg/ano) — a diferença é impressionante.

Programa principal

```

1 // Exercício 21.17 - main_EmissaoCarbono.cpp
2 #include <iostream>
3 #include <iomanip>
4 #include <vector>
5 #include <memory>
6 #include "EmissaoCarbono.h"

```

```

7  #include "Predio.h"
8  #include "Carro.h"
9  #include "Bicicleta.h"
10 using namespace std;
11
12 int main() {
13     cout << fixed << setprecision(2);
14
15     vector<unique_ptr<EmissaoCarbono>> objetos;
16     objetos.push_back(unique_ptr<EmissaoCarbono>(
17         new Predio("Av. Paulista, 1000 - Sao Paulo", 50000)));
18     objetos.push_back(unique_ptr<EmissaoCarbono>(
19         new Predio("Rua Augusta, 200 - Sao Paulo", 12000)));
20     objetos.push_back(unique_ptr<EmissaoCarbono>(
21         new Carro("Honda Civic 1.5", 15000, 12)));
22     objetos.push_back(unique_ptr<EmissaoCarbono>(
23         new Carro("Toyota Hilux 2.8", 20000, 8)));
24     objetos.push_back(unique_ptr<EmissaoCarbono>(
25         new Bicicleta("Caloi 10", 15)));
26
27     cout << "=== Relatorio de Emissao de Carbono ===\n\n";
28
29     double total = 0;
30     for (const auto &obj : objetos) {
31         double e = obj->retEmissaoCarbono();    // POLIMORFICO!
32         cout << obj->descricao() << "\n"
33             << "    -> " << e << " kg CO2/ano\n\n";
34         total += e;
35     }
36
37     cout << "=====\n";
38     cout << "TOTAL DE EMISSOES: " << total << " kg CO2/ano\n";
39     cout << "                = " << (total / 1000) << " toneladas CO2/ano\n";
40     cout << "=====\n";
41     return 0;
42 }

```

Execução

Saída da execução

```

=== Relatorio de Emissao de Carbono ===

Predio em Av. Paulista, 1000 - Sao Paulo (consumo 50000 kWh/ano)
    -> 20000.00 kg CO2/ano

Predio em Rua Augusta, 200 - Sao Paulo (consumo 12000 kWh/ano)
    -> 4800.00 kg CO2/ano

Carro Honda Civic 1.5 (15000 km/ano, 12 km/L)
    -> 2875.00 kg CO2/ano

Carro Toyota Hilux 2.8 (20000 km/ano, 8 km/L)
    -> 5750.00 kg CO2/ano

Bicicleta Caloi 10 (vida util 15 anos)
    -> 16.67 kg CO2/ano

=====
TOTAL DE EMISSOES: 33441.67 kg CO2/ano
                = 33.44 toneladas CO2/ano

```

Conferência matemática

- Predio Av. Paulista: $50000 \times 0,4 = 20\,000$ kg.
- Predio Rua Augusta: $12000 \times 0,4 = 4\,800$ kg.
- Honda Civic: $\frac{15000}{12} \times 2,3 = 1250 \times 2,3 = 2\,875$ kg.
- Toyota Hilux: $\frac{20000}{8} \times 2,3 = 2500 \times 2,3 = 5\,750$ kg.
- Bicicleta: $\frac{250}{15} \approx 16,67$ kg.
- **Total:** $20000 + 4800 + 2875 + 5750 + 16,67 = 33\,441,67$ kg $\approx 33,44$ toneladas.

Pontos técnicos do código

- **EmissaoCarbono** é *puramente abstrata*. Contém apenas duas funções virtuais puras (`retEmissaoCarbono` e `descricao`) e um destrutor virtual vazio. Nenhum dado-membro. É o equivalente C++ de uma *interface* em Java/C#.
- **Herança sem dados compartilhados**. As três derivadas não herdam nem nome, nem endereço, nem qualquer outro dado. Compartilham apenas o *contrato* de fornecer `retEmissaoCarbono()`.
- **Loop polimórfico** (linha 22 do `main`): `obj->retEmissaoCarbono()` chama a versão correta para cada tipo, sem qualquer `switch` ou checagem de tipo.
- **unique_ptr** garante destruição automática. `virtual ~EmissaoCarbono()` garante que o destrutor da derivada seja chamado corretamente através do ponteiro à base.

2 Reflexão sobre o impacto ambiental

Reflexão — Fazendo a Diferença

O que esses números significam?

Um `ToyotaHilux` (5 750 kg CO₂/ano) sozinho emite *345 vezes* mais que uma `Bicicleta` de vida útil 15 anos (16,67 kg/ano). Mesmo um `Honda Civic` (2 875 kg/ano) emite *172 vezes* mais que a bicicleta.

Comparativos para contexto:

- Emissão média per capita anual:
 - Brasil: $\approx 2,3$ toneladas (uma das mais baixas, em função da matriz elétrica hidrelétrica).
 - EUA: $\approx 14,9$ toneladas.
 - Mundo: $\approx 4,7$ toneladas.
- **Meta para 1,5°C** (Acordo de Paris): per capita global deve cair para $\approx 2,3$ toneladas até 2030 e ≈ 1 tonelada até 2050.

Os 33,44 toneladas do nosso exemplo (5 entidades) equivalem a aproximadamente o dobro do que *1 norte-americano médio* emite individualmente em um ano. Isso *seria* a pegada de combinação familiar mais 2 carros + 2 prédios pequenos.

Limitações do modelo

- **Fator de eletricidade depende da região.** 0,4 kg/kWh é a média americana. No Brasil, com matriz limpa, seria $\approx 0,07$ kg/kWh; em países que ainda usam predominantemente carvão (Índia, Polônia), pode chegar a 0,9 kg/kWh.
- **Carro: emissão varia com tipo de combustível.** Diesel emite $\approx 2,68$ kg/L (mais denso); etanol queimado em motor flex tem balanço quase-zero (o CO₂ liberado foi absorvido pela cana durante o crescimento); veículos elétricos jogam a emissão para a fonte de geração elétrica.
- **Bicicleta: a fabricação varia.** Quadros de alumínio são mais “pesados” em carbono; quadros de aço, mais leves; quadros de fibra de carbono, surpreendentemente, têm pegada significativa por causa do processamento.
- **Não inclui ciclo de vida completo.** Não consideramos emissões na fabricação de carros ou de prédios (que tipicamente são *enormes*, dezenas de toneladas por carro e milhares de toneladas por prédio — amortizadas em sua vida útil).

Extensões didáticas possíveis

- Adicionar `Aviao` (vai *disparar* qualquer outra emissão — voos têm alto custo de CO₂).
- Adicionar `CarroEletrico` com parâmetros distintos (kWh/100km, fator da rede).
- Acrescentar `Floresta` com método `retSequestroCarbono()` para subtrair emissões.
- Computar emissões mensais ao invés de anuais.

Observação de Engenharia de Software

A escolha de uma *interface pura* (`EmissaoCarbono`) em vez de uma classe-base com dados/código compartilhado é particularmente apropriada quando os tipos a unificar não têm essência comum. Prédios, carros e bicicletas não são “tipos de emissão de carbono” — são *entidades que casualmente emitem carbono*. A diferença é sutil mas crucial: usar uma classe-base com atributos comuns introduziria acoplamento artificial.

Padrão Strategy: este exercício é um exemplo natural do *padrão Strategy* do livro do GoF: encapsular uma família de algoritmos (aqui, modos de calcular emissão) e torná-los intercambiáveis. Cada classe “é” uma estratégia de cálculo, e o código cliente não distingue.

Encerramento

Este Fazendo a Diferença sintetiza a lição mais profunda do Capítulo 21: *polimorfismo é uma ferramenta de unificação*. Permite tratar uniformemente coisas radicalmente diferentes, desde que compartilhem um contrato bem definido. Aplicada ao problema climático, a técnica oferece uma maneira limpa de modelar e somar emissões de fontes heterogêneas — **Prédios**, veículos, processos industriais. Em código de produção real (calculadoras de pegada de carbono, sistemas de ESG corporativo), essa é exatamente a estrutura usada.